CtrlPay — Let AI Control the Flow

> **Stop Paying Blindly. Let AI Control the Flow.** A **proactive, defensive** personal-finance agent for university students. Built for the **2026 Shenzhen FinTechathon — Topic A**.

CtrlPay does not just *report* what you spent. It **interrupts** a harmful purchase before it happens, **quarantines** impulse buys, **sweeps** zombie subscriptions, and **proves every ringgit** with a tamper-evident audit trail.

> ⚠️ **Sandbox.** No real funds move. Every balance, vault, escrow and reward is a labelled local ledger bucket. Not a bank, not financial advice.

Live demo: https://ctrlpay-new-bee4.vercel.app **Source:** https://github.com/zhangze1007/Yi-HUi **Demo account:** none required — the judge sandbox opens with a fresh synthetic student profile.

2026 Shenzhen FinTechathon — Submission Checklist & Evidence Index

#	Deliverable	Where
01	Technical documentation (architecture, algorithms, security design)	`docs/TECHNICAL_DOCUMENTATION.md`
04	Source code & tests	`README.md`, `tests/`, `THIRD_PARTY_LICENSES.md`
05	Security self-assessment report	`SECURITY_REPORT.md`
06	Execution evidence (sandbox operation logs)	`docs/SANDBOX_EXECUTION_EVIDENCE.md`, `docs/EXECUTION_EVIDENCE_SANDBOX_LOGS.json`

The execution evidence is produced deterministically by `npm run generate:evidence` (it runs the 8-step judge workflow against the real engine, store and audit logger). The same evidence can be exported live from the in-app **Audit Log** → **Export evidence** button.

v2.5 highlights (judge quick map)

- **Multi-currency / PPP display layer** — 5 currencies (MYR · CNY · HKD · SGD · USD) switch instantly; the ledger and engine stay 100% MYR (`lib/currency-ppp.ts`).
- **Back-Tap quick logging** — `?trigger=backtap` deep link, sandbox button, or double-tap `B`; live Dynamic Daily Allowance preview (`components/quick-log-backtap-modal.tsx`).
- **Zero-knowledge passkey gate** — Face ID / fingerprint simulation with FIDO2 authorization refs on T3 signatures (`components/biometric-passkey-gate.tsx`).
- **Assisted cancellation sheet** — whitelisted deep link + in-app email preview + two-stage PendingSweep (`components/subscription-cancel-email-sheet.tsx`).

Quick start

```
npm install
cp .env.example .env.local      # then edit .env.local
npm run dev                     # http://localhost:3000
```

Environment variables (`.env.local`)

Variable	Required	Purpose
`GROQ_API_KEY`	For the live AI	Direct Groq key for the agent chat, OCR receipt check and payment advice. Without it the AI routes return **503** and the dashboard, Zombie Radar, timeline and audit log still work fully.
`AI_GATEWAY_API_KEY`	Optional fallback	Used only when `GROQ_API_KEY` is absent. Requests go through Vercel AI Gateway, pinned to the Groq provider, with the same `openai/gpt-oss-120b` model.
`GROQ_MODEL`	Optional	Overrides the default `openai/gpt-oss-120b`. Retired Llama 3.x overrides automatically fall back to the current model.
`CTRLPAY_SIGNING_SECRET`	Production only	Server secret used to HMAC-SHA256–sign proposals. A labelled dev fallback is used when unset.

Verify the build

```
npm run typecheck           # tsc --noEmit (strict)
npm test                    # vitest (unit + compliance)
npm run build               # next build (production)
npm run lint                # next lint
npm run verify:sdk-version  # AI SDK v4 pin guard (no v5 identifiers)
npm run generate:evidence   # regenerate docs/SANDBOX_EXECUTION_EVIDENCE.md (deterministic)
```

Architecture

Two brains, one rule: **the model never computes money.**

```
LLM (Groq GPT-OSS 120B) ──extracts intent + identifiers only──▶  tool call
                                                                    │
SERVER  lib/quant-engine.ts ──deterministically computes ALL figures──▶ SIGNED proposal
                                                                    │
                                                  USER ──approve / modify / decline──▶ the exact
proposal
```

- **Numbers** come from `lib/quant-engine.ts` (historical-bootstrap **Cash-Flow-at-Risk**, **Basel III 95% Expected Shortfall**, an **intra-month liquidity barrier** for non-sufficient-funds dips, sinking fund, dynamic daily allowance), never from the model.

- **Authority** comes from the signed session cookie (`lib/session.ts`), never the request body.
- **Guardrails** are three-stage: pre-LLM PII masking → per-user tenant isolation → strict Zod schemas + a post-LLM RM **number verifier** (`lib/privacy-guard.ts`).
- **Audit** is an append-only **hash chain** (`lib/audit-logger.ts`).

Mobile app (Phase 8)

The student app ships in three synchronized layout modes over one store — **App + Judge Studio** (default), **Full App Focus**, and **Expanded Dashboard** — so judges can watch the phone and the audit chain at the same time. The phone shell (`components/mobile-app-shell.tsx`) is a faithful port of the team's earlier e-wallet prototype (`docs/legacy/duoMoney_Plus_v36.html`) onto CtrlPay's ledger: 5-tab bottom navigation, balance hero, needs-vs-wants ring, dual spend limits, **AI mandate sliders** (clamped RM 10–1,000 / 1–30 days, T3 stays click-only), the 12:00 daily summary, the Month-End Roast (Gentle / Spicy) and the audit-chain badge.

Phase 8 hardening on top of the CFaR engine (all deterministic, all unit-tested):

Metric	Meaning
`es95`	95% **Expected Shortfall** — mean of the worst 5% of month-end shortfalls (coherent, Artzner et al. 1999; `es95 ≥ cfar95` proven per scenario)
`intraMonthLiquidityBreachProb`	P(running balance < RM 0 mid-month) — the NSF/liquidity-barrier view, timing-aware (day-25 allowance, unpaid bills, daily spend)
`lowestProjectedBalanceRM`	5th-percentile path minimum (the "how low can it dip" figure)
`stratifiedShortfallProbability`	Weekday/weekend stratified bootstrap (Künsch 1989) — weekends are structurally heavier
`presentBiasInflationRatio`	1/β = **1.43×** Laibson (1997) present bias — the exact markup a 24h quarantine cancels
`riskElasticRewardRM`	Commitment reward that scales with simulated risk while staying capped at RM 15 and pool-bounded
maturity gate	Non-monthly charges due ≤ 7 days must be held as **spot cash** (RM 180/year gym ≠ RM 15/month)
clearing telemetry	±5% FX/scheme drift, ±3-day settlement lag, ISO 20022 `pain.009` mandate category

Permission tiers

Tier	Meaning	Examples
T0	Read-only analysis, always allowed	`run_what_if_simulation` , `get_budget_insights` , `analyze_zombies`
T1	Reversible, no money state	`flag_suspected_zombie`
T2	Prepares an external action	`draft_cancellation`

Tier	Meaning	Examples
T3	Needs **Master ON + a per-action click**	`propose_quarantine`, `propose_escrow`, `propose_sweep`

The Master AI Autonomy Switch is **default OFF**. T0 safety analysis still works off — so the protective core is **never paywalled or gated** (enforced by `tests/compliance.test.ts`).

Key source files

Path	Role
`lib/quant-engine.ts`	All money math (CFaR, escrow floor, quarantine reward, amortization)
`lib/store.ts`	Zustand sandbox state + event-driven clock (Section 5.1 resolution order)
`lib/permissions.ts`	T0–T3 tiers, `assertTier`, click-authorization gate
`lib/privacy-guard.ts`	PII masking + `verifyNumbers`
`lib/audit-logger.ts`	Hash-chained, append-only audit log
`lib/proposals.ts`	Server-side proposal creation / decision (signs + re-verifies)
`app/api/chat/route.ts`	Groq Chat Completions agent loop, tool registry, AI SDK v4 data stream
`lib/groq-client.ts`	Server-only Groq client, model resolution/retirement guard, OpenAI-compatible tool schemas
`app/api/ai-analyze/route.ts`	Advisory OCR red-flag check + payment-interceptor advice
`app/api/proposals/**`	Signed proposal + decision routes
`lib/tool-resolution.ts`	Contract-card resolution + dangling-tool-call guard
`components/agent-chat.tsx`	`useChat` (v4) + the three interactive contract cards
`components/subscription-timeline.tsx`	30-day strip, amortization toggle, maturity-gate banner, fast-forward
`components/mobile-app-shell.tsx`	Phase 8 phone shell — 5 tabs, balance hero, limits, mandate sliders, roast, audit badge
`components/cfar-stress-tester.tsx`	Interactive What-If CFaR/ES95/NSF tester (sliders + presets)
`components/risk-tail-strip.tsx`	Shared ES95 / NSF / latency / present-bias telemetry strip

5-minute demo script

> **Before you begin:** every contract action (Quarantine / Escrow / Sweep) is **T3**, which requires the **Master AI Autonomy Switch to be ON** *and* an explicit click. Steps 1–2 show it OFF; **turn it ON in the Simulation Toolbar before step 3.**

1. **Open the dashboard.** Master Switch is **OFF** → the toolbar shows `T0 ADVISORY ONLY`, so only analysis is available. This demonstrates default-OFF safety.
2. **Click `Simulate Shopee RM 180`.** Auto CFaR runs, then the **intercept modal** appears with the **shortfall probability** and duty-of-care figures. (A low-risk purchase would show no contract.)
3. **Turn the Master Switch ON**, then click `24h Quarantine` → the purchase becomes `LOCKED_24H` and a reward is reserved from the `rewardPool`.
4. **Click `+24 Hours`.** The quarantine flips to `READY_TO_DECIDE`. Click `Abandon Purchase & Claim Reward` — the reward is paid **from the rewardPool**. (Claiming before +24h is impossible; the button is disabled and titled *"Available after +24h"*.)
5. **Zombie Radar** flags **Spotify** as paid at full price while a student plan exists → click `Student downgrade` → enter **any 6-digit OTP** → use the **registry deep link** or the `mailto:` **draft** → **confirm** → a **PendingSweep** is recorded.
6. **Click `Jump to Month-End`.** The sweep is **verified** (the old charge disappeared), the **Defense Vault is credited**, and the **Month-End Roast** is shown (Gentle / Spicy).
7. **Open the Audit Drawer.** Show `data_accessed`, `authorization_ref`, `proposal_hash` and the **chain-verified** badge — the whole session is tamper-evident.

Security & compliance

- 🔐 **Security Self-Assessment Report:** `SECURITY_REPORT.md` (Section 10)
- 💉 **Red-team test matrix (T-01 … T-10):** `docs/RED_TEAM_TEST_MATRIX.md`
- 📦 **Third-party licenses:** `THIRD_PARTY_LICENSES.md`
- 📐 **Design blueprint:** `plan.md`

Hardening highlights: strict CSP (`default-src 'self'`, no third-party scripts, no pop-up ads), HSTS, `X-Frame-Options: DENY`, `noopener noreferrer` on all external links, a hardcoded HTTPS registry whitelist (the LLM can never supply a URL), and **no autonomous credentials** — CtrlPay holds no OAuth write access.

Platform-layer limits (disclosed, not claimed)

A web app cannot detect hostile Wi-Fi or block pop-ups from other apps; those are OS/browser responsibilities. The sandbox session is not strongly authenticated (a signed cookie stands in for real login), and the audit log lives in memory/ `localStorage`. See the report's *Residual Risk* column for the production path (step-up auth, device binding, WORM storage, eKYC, certificate pinning).

Multi-turn tool-call integrity

The three contract tools are **client-resolved** (no server `execute`): the card must call `addToolResult` for its `toolCallId` whatever the user does — **approve, modify, decline, or clear the chat**. `lib/tool-resolution.ts` centralises these payloads and `tests/tool-resolution.test.ts` proves no contract call is left dangling. `useChat` runs with `maxSteps: 5` so the model can continue after a decision.

CtrlPay is a competition sandbox. Merchant names are labels for mock data; no logos or brand assets are included.